

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
КРЕМЕНЧУЦЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ МИХАЙЛА ОСТРОГРАДСЬКОГО
НАВЧАЛЬНО-НАУКОВИЙ ІНСТИТУТ ЕЛЕКТРИЧНОЇ ІНЖЕНЕРІЇ
ТА ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ



МЕТОДИЧНІ ВКАЗІВКИ
ЩОДО ВИКОНАННЯ ЛАБОРАТОРНИХ І САМОСТІЙНОЇ РОБІТ
З НАВЧАЛЬНОЇ ДИСЦИПЛІНИ
«ОБ'ЄКТНО-ОРІЄНТОВАНЕ ПРОГРАМУВАННЯ»
ДЛЯ ЗДОБУВАЧІВ ВИЩОЇ ОСВІТИ (ПЕРШОГО) БАКАЛАВРСЬКОГО
РІВНЯ ДЕННОЇ ФОРМИ НАВЧАННЯ
ЗІ СПЕЦІАЛЬНОСТІ 123 – «КОМП'ЮТЕРНА ІНЖЕНЕРІЯ»
ОСВІТНЬО-ПРОФЕСІЙНОЇ ПРОГРАМИ «КОМП'ЮТЕРНА ІНЖЕНЕРІЯ»
(ЧАСТИНА І)

КРЕМЕНЧУК 2024

Методичні вказівки щодо виконання лабораторних і самостійної робіт з навчальної дисципліни «Об'єктно-орієнтоване програмування» для здобувачів вищої освіти (першого) бакалаврського рівня денної форми навчання зі спеціальності 123 – «Комп'ютерна інженерія» освітньо-професійної програми «Комп'ютерна інженерія» (частина I)

Укладач к. т. н., доц. Д. Г. Мамчур

Рецензент д. т. н., проф. А. Л. Перекрест

Кафедра комп'ютерної інженерії та електроніки

Затверджено методичною радою Кременчуцького національного університету імені Михайла Остроградського

Протокол №10 від 26.06.2024 року

Голова методичної ради



проф. Віктор КОСТІН

ЗМІСТ

Вступ.....	4
1 Вимоги до оформлення звітів з лабораторних робіт.....	6
2 Перелік лабораторних робіт.....	7
Лабораторна робота № 1 Основи синтаксису Java	7
Лабораторна робота № 2 Робота з класами та об'єктами	12
Лабораторна робота № 3 Методи та рекурсія	19
Лабораторна робота № 4 Наслідування та поліморфізм	24
Лабораторна робота № 5 Обробка винятків у Java	33
Лабораторна робота № 6 Робота з колекціями (List, Set, Map).....	40
Лабораторна робота № 7 Робота з файлами та серіалізація	47
3 Критерії оцінювання знань студентів.....	56
Список літератури	58
Додаток А Зразок оформлення титульної сторінки звіту виконання індивідуального завдання.....	60

ВСТУП

Перша частина методичних вказівок містить перші сім лабораторних робіт, які входять до першого і другого змістовних модулів «Основи Java та ООП» та «Основні принципи ООП» відповідно, і містять лабораторні роботи, присвячені базовим питанням синтаксису Java, основних програмних конструкцій, будівельних блоків об'єктно-орієнтованих мов програмування, обробці винятків та роботі з файловим введенням-виведенням.

Мета лабораторних робіт – узагальнити знання в межах базового розуміння основ програмування, зокрема, з використанням об'єктно-орієнтованого підходу, та набути навичок практичного застосування отриманих знань.

Методичні вказівки для студентів уміщують короткі теоретичні відомості, приклади розв'язування задач, вимоги щодо оформлення звіту індивідуальних завдань з лабораторних робіт.

Метою навчальної дисципліни є формування у студентів фундаментальних знань і практичних навичок об'єктно-орієнтованого програмування (ООП) на мові Java, що дозволять їм розробляти ефективні, масштабовані та підтримувані програмні рішення.

Завданням навчальної дисципліни є застосування базових знань та засобів програмування для створення прикладних програмних застосунків на з дотриманням принципів ООП з їх практичним використанням.

У результаті вивчення навчальної дисципліни студент повинен ***отримати досвід з таких компетентностей.***

ЗК 1. Здатність до абстрактного мислення, аналізу та синтезу.

ЗК 2. Здатність вчитися і оволодівати сучасними знаннями.

ЗК 3. Здатність застосовувати знання у практичних ситуаціях.

ЗК 7. Вміння виявляти, ставити та вирішувати проблеми.

ФК 1. Здатність застосовувати законодавчу та нормативно-правову базу, а також державні та міжнародні вимоги, практики і стандарти з метою здійснення професійної діяльності в галузі комп'ютерної інженерії.

ФК 2. Здатність використовувати сучасні методи і мови програмування для розроблення алгоритмічного та програмного забезпечення.

ФК 3. Здатність створювати системне та прикладне програмне забезпечення комп'ютерних систем та мереж.

ФК11. Здатність оформляти отримані робочі результати у вигляді презентацій, науковотехнічних звітів.

ФК 15. Здатність аргументувати вибір методів розв'язування спеціалізованих задач, критично оцінювати отримані результати, обґрунтовувати та захищати прийняті рішення.

набути навички й уміння.

ПРН 1. Знати і розуміти наукові положення, що лежать в основі функціонування комп'ютерних засобів, систем та мереж.

ПРН 3. Знати новітні технології в галузі комп'ютерної інженерії.

ПРН 11. Вміти здійснювати пошук інформації в різних джерелах для розв'язання задач комп'ютерної інженерії.

ПРН 12. Вміти ефективно працювати як індивідуально, так і у складі команди.

ПРН 16. Вміти оцінювати отримані результати та аргументовано захищати прийняті рішення.

ПРН 18. Використовувати інформаційні технології для ефективного спілкування на професійному та соціальному рівнях.

ПРН 21. Якісно виконувати роботу та досягати поставленої мети з дотриманням вимог професійної етики.

1 ВИМОГИ ДО ОФОРМЛЕННЯ ЗВІТІВ

3 ЛАБОРАТОРНИХ РОБІТ

Кожна лабораторна робота містить індивідуальне завдання. Кожна лабораторна робота має розглядатися як окремий пакет у загальному проєкті лабораторних робіт з назвою `lab<номер лаб.>` під контролем системи контролю версій Git, розміщений у репозиторію з ім'ям `oopjava_<student_lastname>` на GitHub, до якого надається доступ викладачу. Папка має містити всі необхідні дані й додаткові файли. Звіт з роботи оформлюється у вигляді *.docx-файлу.

Виконана робота подається на перевірку через онлайн-платформу krnu.org. Студент також має прикріпити посилання на свій GitHub-репозиторій.

2 ПЕРЕЛІК ЛАБОРАТОРНИХ РОБІТ

Лабораторна робота № 1

Тема. Основи синтаксису Java

Мета: ознайомлення з базовим синтаксисом мови програмування Java та набуття практичних навичок написання простих програм у середовищі розробки IntelliJ IDEA.

Короткі теоретичні відомості

Java є об'єктно-орієнтованою мовою програмування, яка має чіткий та структурований синтаксис. Основні елементи синтаксису Java включають:

- структура програми: кожна програма на java складається з одного або кількох класів. кожен клас містить методи, які визначають поведінку об'єктів цього класу. точкою входу в програму є метод `main()`, який має наступний сигнатурний вигляд: `public static void main(stringargs);`
- ідентифікатори: імена класів, методів, змінних тощо називаються ідентифікаторами. вони можуть складатися з літер, цифр, символів підкреслення (`_`) та долара (`$`), але не можуть починатися з цифри. java є чутливою до регістру, тому `myvariable` та `Myvariable` вважаються різними ідентифікаторами;
- ключові слова: це зарезервовані слова, які мають спеціальне значення в мові java (наприклад, `class`, `public`, `static`, `void`, `int`, `if`, `for` тощо);
- типи даних: java підтримує примітивні типи даних (наприклад, `int`, `double`, `boolean`, `char`) та посилальні типи даних (наприклад, класи, інтерфейси, масиви);
- змінні: змінні використовуються для зберігання даних. перед використанням змінну необхідно оголосити, вказавши її тип та ім'я;
- оператори: java надає різноманітні оператори для виконання операцій над даними (наприклад, арифметичні оператори `+`, `-`, `*`, `/`, `%`; оператори порівняння `==`, `!=`, `>`, `<`, `>=`, `<=`; логічні оператори `&&`, `||`, `!`);

- керуючі структури: для керування порядком виконання інструкцій використовуються керуючі структури, такі як умовні оператори (if, else, else if) та цикли (for, while, do-while);
- коментарі: коментарі використовуються для пояснення коду та ігноруються компілятором. в java існують однорядкові коментарі (//) та багаторядкові коментарі (/* ... */);
- виведення на консоль: для виведення інформації на консоль використовується метод `system.out.println()`.

Порядок виконання роботи

1. Запуск IntelliJ IDEA. Запустіть середовище розробки IntelliJ IDEA Community Edition.

2. Створення нового проекту:

- на початковому екрані оберіть опцію **New Project**;
- у вікні **New Project**, в лівій частині оберіть **Java**;
- переконайтеся, що обрано відповідний **Project SDK**. Якщо SDK не налаштовано, натисніть **Add SDK...** та вкажіть шлях до встановленої Java Development Kit (JDK);
- натисніть кнопку **Next**;
- на наступному екрані (Template), якщо ви не використовуєте шаблони, просто натисніть **Next**;
- введіть назву проекту у поле **Project name** (наприклад, LabsOOP);
- вкажіть шлях до каталогу, де буде збережено проект, у полі **Project location**;
- натисніть кнопку **Create**.

3. Створення нового класу:

- у вікні **Project** (зазвичай розташоване зліва), розгорніть створений проект (LabsOOP);

- перейдіть до каталогу `src` (якщо він є) або безпосередньо до кореня проекту;
- натисніть правою кнопкою миші на каталозі `src` (або корені проекту);
- у контекстному меню оберіть **New -> Java Class**;
- у вікні **Create New Class** введіть назву класу (наприклад, `HelloWorld`) та натисніть **OK**.

4. Написання коду. У вікні редактора коду введіть наступний код:

```
public class HelloWorld {
    public static void main(String args) {
        System.out.println("Привіт, Java!"); // Виведення тексту на консоль
    }
}
```

5. Запуск програми:

- натисніть правою кнопкою миші на редакторі коду в будь-якому місці;
- у контекстному меню оберіть **Run 'HelloWorld.main()'**;
- або натисніть зелену стрілку запуску, яка може бути розташована у верхній панелі інструментів або зліва від рядка з методом `main()`.

6. Перегляд результату. Результат виконання програми буде відображено у вікні `Run` (зазвичай розташоване в нижній частині екрану). Ви повинні побачити текст `Привіт, Java!`.

7. Експерименти з синтаксисом. Спробуйте змінити код, додаючи оголошення змінних різних типів, виконуючи прості арифметичні операції та виводячи їх результати на консоль. Наприклад:

```
public class BasicSyntax {
    public static void main(String args) {
        int number1 = 10;
        int number2 = 5;
        int sum = number1 + number2;
        System.out.println("Сума: " + sum);

        double pi = 3.14159;
        System.out.println("Значення PI: " + pi);
    }
}
```

```

        boolean isTrue = true;
        System.out.println("Логічне значення: " + isTrue);

        String message = "Це рядок.";
        System.out.println(message);
    }
}

```

8. Виконайте індивідуальне завдання згідно варіантом, отриманим від викладача.

Індивідуальне завдання

Напишіть програму на Java, яка виконує наступні дії (для кожного варіанту змініть вхідні дані).

Варіанти індивідуальних завдань.

1. Обчислює суму двох цілих чисел.
2. Обчислює різницю двох дійсних чисел.
3. Обчислює добуток двох цілих чисел.
4. Обчислює частку від ділення одного дійсного числа на інше.
5. Виводить на консоль повідомлення "Hello, World!".
6. Виводить на консоль ваше ім'я.
7. Виводить на консоль поточний рік.
8. Виводить на консоль результат порівняння двох цілих чисел на рівність.
9. Виводить на консоль результат порівняння двох дійсних чисел на нерівність.
10. Виводить на консоль результат перевірки, чи є задане ціле число парним.
11. Виводить на консоль результат перевірки, чи є задане ціле число додатним.
12. Виводить на консоль результат логічної операції AND між двома булевими значеннями.
13. Виводить на консоль результат логічної операції OR між двома булевими значеннями.

14. Виводить на консоль результат логічної операції NOT для заданого булевого значення.
15. Виводить на консоль площу прямокутника за заданими довжиною та шириною (цілі числа).
16. Виводить на консоль периметр прямокутника за заданими довжиною та шириною (дійсні числа).
17. Виводить на консоль площу кола за заданим радіусом (дійсне число).
18. Виводить на консоль довжину кола за заданим радіусом (ціле число).
19. Виводить на консоль середнє арифметичне трьох цілих чисел.
20. Виводить на консоль максимальне значення з двох дійсних чисел.
21. Виводить на консоль мінімальне значення з двох цілих чисел.
22. Виводить на консоль результат округлення заданого дійсного числа до найближчого цілого.
23. Виводить на консоль залишок від ділення одного цілого числа на інше.
24. Виводить на консоль результат піднесення заданого цілого числа до квадрату.
25. Виводить на консоль результат перевірки, чи є перше задане ціле число більшим за друге.
26. Виводить на консоль результат перевірки, чи є перше задане дійсне число меншим або рівним другому.
27. Виводить на консоль конкатенацію двох заданих рядків.
28. Виводить на консоль довжину заданого рядка.
29. Виводить на консоль перший символ заданого рядка.
30. Виводить на консоль повідомлення, що складається з двох заданих рядків, розділених пробілом.

Контрольні питання

1. Яка структура простої програми на Java? Що таке клас та метод main()?
2. Що таке ідентифікатор в Java? Які існують правила для їх написання?

3. Які основні примітивні типи даних існують в Java? Наведіть приклади.
4. Як оголосити та ініціалізувати змінну в Java?
5. Які основні арифметичні оператори підтримує Java?
6. Для чого використовуються керуючі структури if-else та for? Наведіть приклади.
7. Як вивести інформацію на консоль в Java? Який метод для цього використовується?

Література: [1–4, 6, 8, 14, 15].

Лабораторна робота № 2

Тема. Робота з класами та об'єктами

Мета: опанування базових навичок створення класів, об'єктів, оголошення атрибутів та методів.

Короткі теоретичні відомості

Об'єктно-орієнтоване програмування (ООП) є парадигмою програмування, яка базується на концепції "об'єктів", які можуть містити дані у формі полів (атрибутів або властивостей) і код у формі процедур (методів або функцій).

Клас є кресленням або шаблоном для створення об'єктів. Він визначає структуру та поведінку, які будуть мати всі об'єкти цього класу. Клас описує набір атрибутів (змінних-членів) та методів (функцій-членів), які характеризують об'єкти цього класу.

Об'єкт є екземпляром класу. Коли створюється об'єкт, йому виділяється пам'ять, і він отримує всі атрибути та методи, визначені у його класі.

Атрибути (поля) – це змінні, які зберігають дані, що характеризують стан об'єкта.

Методи – це функції, які визначають поведінку об'єкта, тобто дії, які об'єкт може виконувати.

Конструктор – це спеціальний метод класу, який викликається автоматично при створенні нового об'єкта. Його основне призначення – ініціалізувати атрибути об'єкта.

Порядок виконання роботи

1. Запустіть IntelliJ IDEA Community Edition.
2. Створіть новий проєкт (якщо не створено у попередній роботі):
 - у вікні "Welcome to IntelliJ IDEA" оберіть "New Project";
 - у вікні "New Project" оберіть "Java" у лівій панелі;
 - переконайтеся, що у полі "Project SDK" обрано відповідний Java Development Kit (JDK). Якщо ні, натисніть "Add JDK..." та вкажіть шлях до встановленого JDK;
 - натисніть кнопку "Next";
 - зніміть позначку з "Create project from template" (якщо вона встановлена);
 - натисніть кнопку "Next";
 - у полі "Project name" введіть назву проєкту, наприклад, "LabsOOP";
 - у полі "Location" вкажіть шлях до папки, де буде збережено проєкт;
 - натисніть кнопку "Create".
3. Створіть новий пакет:
 - у вікні "Project" (зазвичай розташоване зліва) знайдіть папку src;
 - правою кнопкою миші клацніть на папці src;
 - у контекстному меню оберіть "New" -> "Package";
 - у вікні "New Package" введіть назву пакету, наприклад, lab02, та натисніть клавішу "Enter".
4. Створіть новий клас:
 - у вікні "Project" (зазвичай розташоване зліва) знайдіть папку src.
 - правою кнопкою миші клацніть на пакеті, що створили у п. 3.
 - у контекстному меню оберіть "New" -> "Java Class".

- у вікні "New Java Class" введіть назву класу, наприклад, Rectangle, та натисніть клавішу "Enter".

5. Оголошення атрибутів класу:

- у файлі Rectangle.java оголосіть два атрибути для представлення ширини та висоти прямокутника:

```
public class Rectangle {  
    public double width;  
    public double height;  
}
```

6. Створення конструктора класу:

- додайте конструктор, який приймає значення ширини та висоти як аргументи та ініціалізує відповідні атрибути:

```
public class Rectangle {  
    public double width;  
    public double height;  
  
    public Rectangle(double width, double height) {  
        this.width = width;  
        this.height = height;  
    }  
}
```

7. Створення методів класу:

- додайте методи для обчислення площі та периметра прямокутника:

```
public class Rectangle {  
    public double width;  
    public double height;  
  
    public Rectangle(double width, double height) {  
        this.width = width;  
        this.height = height;  
    }  
  
    public double calculateArea() {  
        return width * height;  
    }  
}
```

```

    public double calculatePerimeter() {
        return 2 * (width + height);
    }
}

```

8. Створення головного класу (для тестування):

- у вікні "Project" знову клацніть правою кнопкою миші на папці src та оберіть "New" -> "Java Class";
- введіть назву класу, наприклад, Main, та натисніть "OK";
- додайте метод main до цього класу:

```

public class Main {
    public static void main(String args) {
        // Створення об'єкта класу Rectangle
        Rectangle myRectangle = new Rectangle(5.0, 10.0);

        // Виклик методів об'єкта
        double area = myRectangle.calculateArea();
        double perimeter = myRectangle.calculatePerimeter();

        // Виведення результатів на консоль
        System.out.println("Площа прямокутника: " + area);
        System.out.println("Периметр прямокутника: " + perimeter);
    }
}

```

9. Запуск програми:

- у вікні редактора коду клацніть правою кнопкою миші на файлі Main.java та оберіть "Run 'Main.main()'";
- у вікні "Run" (зазвичай розташоване внизу) ви побачите виведений результат:

Площа прямокутника: 50.0
Периметр прямокутника: 30.0

10. Виконайте індивідуальне завдання згідно варіантом, отриманим від викладача.

Індивідуальне завдання

Створіть клас, що описує заданий об'єкт, з відповідними атрибутами та методами для виконання зазначених дій. Для кожного завдання необхідно:

- визначити необхідні атрибути класу;
- створити конструктор для ініціалізації атрибутів;
- реалізувати методи, що виконують задані дії;
- у головному класі Main створити екземпляр розробленого класу та продемонструвати роботу його методів.

Варіанти індивідуальних завдань.

1. Створити клас Книга з атрибутами: назва, автор, рік видання. Додати метод getInfo(), який повертає рядок з інформацією про книгу.
2. Створити клас Студент з атрибутами: ім'я, прізвище, курс. Додати метод getFullName(), який повертає повне ім'я студента.
3. Створити клас Автомобіль з атрибутами: марка, модель, рік випуску. Додати метод displayInfo(), який виводить на екран інформацію про автомобіль.
4. Створити клас Комп'ютер з атрибутами: процесор, об'єм ОЗУ, об'єм жорсткого диска. Додати метод getDetails(), який повертає рядок з характеристиками комп'ютера.
5. Створити клас Круг з атрибутом: радіус. Додати методи для обчислення площі та довжини кола.
6. Створити клас ПрямокутнийТрикутник з атрибутами: катет1, катет2. Додати метод для обчислення гіпотенузи.
7. Створити клас БанківськийРахунок з атрибутами: номер рахунку, баланс. Додати методи deposit(amount) для поповнення рахунку та withdraw(amount) для зняття коштів.
8. Створити клас Товар з атрибутами: назва, ціна. Додати метод getPriceWithDiscount(discount) для розрахунку ціни зі знижкою (знижка передається у відсотках).

9. Створити клас Телефон з атрибутами: модель, номер телефону. Додати метод `call(phoneNumber)` для імітації дзвінка (вивести повідомлення на екран).
10. Створити клас Годинник з атрибутами: години, хвилини, секунди. Додати метод `displayTime()` для виведення поточного часу у форматі "HH:MM:SS".
11. Створити клас Куб з атрибутом: довжина сторони. Додати методи для обчислення площі поверхні та об'єму куба.
12. Створити клас ЕлектроннаПошта з атрибутами: адреса відправника, адреса отримувача, тема. Додати метод `send()` для імітації відправки листа (вивести повідомлення на екран).
13. Створити клас Працівник з атрибутами: ім'я, посада, зарплата. Додати метод `getSalaryInfo()` для виведення інформації про зарплату.
14. Створити клас Квадрат з атрибутом: довжина сторони. Додати методи для обчислення площі та периметра квадрата.
15. Створити клас МузичнийТвір з атрибутами: назва, виконавець, жанр. Додати метод `play()` для імітації відтворення твору (вивести повідомлення на екран).
16. Створити клас ГеометричнаФігура з атрибутом: назва. Додати метод `getName()`, який повертає назву фігури. (Створіть окремі класи для кола, прямокутника, трикутника, які наслідують цей клас). Це завдання можна спростити, якщо наслідування ще не вивчалось, просто зробіть окремий клас.
17. Створити клас Фільм з атрибутами: назва, режисер, рік випуску. Додати метод `getDetails()` для виведення інформації про фільм.
18. Створити клас Ліфт з атрибутами: поточний поверх, кількість поверхів. Додати методи `moveUp()` та `moveDown()` для переміщення ліфта на один поверх вгору або вниз.
19. Створити клас Термометр з атрибутом: поточна температура. Додати метод `displayTemperature()` для виведення поточної температури.

20. Створити клас Калькулятор з методами `add(a, b)`, `subtract(a, b)`, `multiply(a, b)`, `divide(a, b)` для виконання основних арифметичних операцій. (Цей клас не потребує атрибутів, але його можна включити для демонстрації методів).

21. Створити клас Флешка з атрибутами: об'єм пам'яті, кількість вільного місця. Додати метод `format()` для імітації форматування (встановити кількість вільного місця рівною об'єму пам'яті).

22. Створити клас Телевізор з атрибутами: марка, поточний канал. Додати методи `nextChannel()` та `previousChannel()` для перемикання каналів.

23. Створити клас ШаховийКінь з атрибутами: поточна позиція (наприклад, у вигляді рядка "A1", "H8"). Додати метод `move(newPosition)` для зміни позиції коня.

24. Створити клас Басейн з атрибутами: довжина, ширина, глибина. Додати метод `calculateVolume()` для обчислення об'єму басейну.

25. Створити клас Ресторан з атрибутами: назва, кількість столиків. Додати метод `reserveTable()` для зменшення кількості вільних столиків.

26. Створити клас Потяг з атрибутами: номер потяга, кількість вагонів. Додати метод `addCarriage()` для збільшення кількості вагонів.

27. Створити клас Велосипед з атрибутами: марка, поточна швидкість. Додати методи `accelerate()` та `brake()` для збільшення та зменшення швидкості.

28. Створити клас Ноутбук з атрибутами: модель, об'єм ОЗУ, об'єм жорсткого диска, час роботи від батареї. Додати метод `displayBatteryLife()` для виведення часу роботи від батареї.

29. Створити клас Магазин з атрибутами: назва, адреса. Додати метод `displayAddress()` для виведення адреси магазину.

30. Створити клас Гравець з атрибутами: ім'я, кількість очок. Додати метод `addPoints(points)` для збільшення кількості очок гравця.

Контрольні питання

1. Що таке клас в об'єктно-орієнтованому програмуванні?

2. Що таке об'єкт? Чим об'єкт відрізняється від класу?
3. Що таке атрибути класу? Яке їх призначення?
4. Що таке методи класу? Яке їх призначення?
5. Що таке конструктор класу? Коли він викликається?
6. Яка різниця між змінними-членами класу та локальними змінними методу?
7. Як створити об'єкт класу в Java? Наведіть приклад..

Література: [1–6, 8, 14, 15].

Лабораторна робота № 3

Тема. Методи та рекурсія

Мета: закріпити практичні навички розробки та використання методів, включаючи рекурсивні методи, в мові програмування Java.

Короткі теоретичні відомості

Методи в Java є блоками коду, які виконують певну дію. Вони дозволяють розбити складну програму на менші, керовані частини, підвищують читабельність та повторне використання коду. Методи можуть приймати параметри (аргументи) та повертати значення.

Основні характеристики методів:

- модифікатори доступу: визначають видимість методу (наприклад, `public`, `private`, `protected`, за замовчуванням);
- тип значення, що повертається: визначає тип даних, який повертає метод (якщо метод нічого не повертає, використовується ключове слово `void`);
- назва методу: ідентифікатор, який використовується для виклику методу;
- список параметрів: визначає типи та назви вхідних даних, які метод приймає (може бути порожнім);
- тіло методу: блок коду, який виконується при виклику методу.

Рекурсія — це метод, який викликає сам себе. Рекурсивні методи використовуються для розв'язання задач, які можна розбити на менші підзадачі,

схожі на початкову. Кожен рекурсивний виклик повинен наближати до базового випадку (умови зупинки), щоб уникнути нескінченної рекурсії.

Основні елементи рекурсивного методу:

- базовий випадок (умова зупинки): умова, за якої рекурсія припиняється і метод повертає певне значення без подальших викликів.
- рекурсивний виклик: виклик самого себе з іншими параметрами, що наближають до базового випадку.

Приклади задач, які зручно розв'язувати за допомогою рекурсії: обчислення факторіалу, чисел Фібоначчі, обхід дерев, тощо.

Порядок виконання роботи

1. Відкрийте проект створений для виконання лабораторних робіт у попередніх завданнях.

2. Створення пакета для лабораторної роботи:

- у вікні "Project" (зазвичай розташоване ліворуч) знайдіть каталог src;
- натисніть правою кнопкою миші на каталозі src;
- оберіть "New" -> "Package";
- введіть назву пакета lab03 та натисніть "ОК".

3. Створення основного класу програми:

- натисніть правою кнопкою миші на створеному пакеті lab03;
- оберіть "New" -> "Java Class";
- введіть назву класу (наприклад, Main) та натисніть "ОК".

4. Реалізація методів та рекурсії:

- відкрийте файл Main.java та напишіть код, що демонструє використання звичайних та рекурсивних методів.

Приклад 1: Звичайний метод для обчислення суми двох чисел:

```
package lab03;
```

```
public class Main {  
    public static void main(String args) {  
        int a = 5;  
        int b = 10;
```

```

        int sum = add(a, b);
        System.out.println("Сума " + a + " та " + b + " дорівнює: " + sum);
    }

    public static int add(int num1, int num2) {
        return num1 + num2;
    }
}

```

Приклад 2: Рекурсивний метод для обчислення факторіалу числа:

```
package lab03;
```

```

public class Main {
    public static void main(String args) {
        int n = 5;
        long factorial = factorial(n);
        System.out.println("Факторіал числа " + n + " дорівнює: " + factorial);
    }

    public static long factorial(int n) {
        if (n == 0) {
            return 1; // Базовий випадок
        } else {
            return n * factorial(n - 1); // Рекурсивний виклик
        }
    }
}

```

5. Виконання програми:

- натисніть правою кнопкою миші на файлі Main.java в вікні "Project".
- оберіть "Run 'Main.main()'";
- перегляньте результат виконання програми у вікні "Run".

6. Виконання індивідуального завдання:

- оберіть свій варіант індивідуального завдання з наведеного нижче списку;
- створіть новий клас (або модифікуйте існуючий Main.java) в пакеті lab03 та реалізуйте необхідні методи (звичайні або рекурсивні) для розв'язання задачі;
- переконайтеся, що ваша програма правильно виконує поставлене завдання.

Індивідуальне завдання

У кожному завданні необхідно створити метод (або декілька методів, включаючи рекурсивні, якщо це доцільно) для виконання вказаної операції. Для кожного завдання необхідно:

- визначити необхідні атрибути класу;
- створити конструктор для ініціалізації атрибутів;
- реалізувати методи, що виконують задані дії;
- у головному класі Main створити екземпляр розробленого класу та продемонструвати роботу його методів.

Варіанти індивідуальних завдань.

1. Написати метод для обчислення площі прямокутника за заданими довжиною та шириною.
2. Написати метод для знаходження найбільшого з трьох заданих чисел.
3. Написати метод для перевірки, чи є задане число парним.
4. Написати метод для обчислення суми всіх чисел у заданому масиві цілих чисел.
5. Написати рекурсивний метод для обчислення n-го числа Фібоначчі.
6. Написати метод для обчислення середнього арифметичного значення елементів у заданому масиві дійсних чисел.
7. Написати метод для знаходження мінімального елемента у заданому масиві цілих чисел.
8. Написати метод для перевірки, чи є заданий рядок паліндромом (читається однаково в обох напрямках).
9. Написати рекурсивний метод для обчислення суми цифр заданого цілого числа.
10. Написати метод для об'єднання двох заданих рядків.
11. Написати метод для знаходження кількості входжень заданого символу в заданий рядок.

12. Написати метод для обчислення степеня числа (без використання вбудованих функцій).
13. Написати рекурсивний метод для знаходження найбільшого спільного дільника (НСД) двох чисел за алгоритмом Евкліда.
14. Написати метод для перевірки, чи є задане число простим.
15. Написати метод для розвороту заданого рядка.
16. Написати метод для знаходження другого за величиною елемента в масиві цілих чисел.
17. Написати метод для перетворення температури з градусів Цельсія в градуси Фаренгейта.
18. Написати рекурсивний метод для друку чисел від 1 до n.
19. Написати метод для видалення всіх пробілів із заданого рядка.
20. Написати метод для знаходження індексу першого входження заданого елемента в масиві (якщо елемент не знайдено, повернути -1).
21. Написати метод для перевірки, чи є заданий рік високосним.
22. Написати рекурсивний метод для обчислення суми елементів масиву цілих чисел.
23. Написати метод для сортування масиву цілих чисел за зростанням (наприклад, методом бульбашки).
24. Написати метод для знаходження кількості слів у заданому рядку.
25. Написати метод для перетворення першої літери кожного слова в рядку на велику.
26. Написати рекурсивний метод для обчислення n-го трикутного числа ($T_n = 1 + 2 + \dots + n$).
27. Написати метод для знаходження частоти кожного символу в заданому рядку.
28. Написати метод для перевірки, чи є один рядок анаграмою іншого.
29. Написати метод для генерації масиву випадкових цілих чисел заданої довжини в заданому діапазоні.

30. Написати рекурсивний метод для знаходження двійкового представлення заданого цілого числа.

Контрольні питання

1. Що таке метод в Java? Яке його призначення?
2. Які існують модифікатори доступу для методів в Java? Чим вони відрізняються?
3. Що таке параметри методу? Як передаються параметри в методи Java?
4. Що таке рекурсія? Наведіть приклади задач, які ефективно розв'язуються за допомогою рекурсії.
5. Які є основні складові частини рекурсивного методу?
6. Яка небезпека використання рекурсії? Як її уникнути?
7. Чим відрізняється звичайний метод від рекурсивного? Коли доцільно використовувати кожен з них?

Література: [1–4, 7, 8, 14, 15].

Лабораторна робота № 4

Тема. Наслідування та поліморфізм

Мета: опанувати практичні навички застосування механізмів наслідування та поліморфізму в мові програмування Java.

Короткі теоретичні відомості

Наслідування (Inheritance) – це механізм, який дозволяє одному класу (класу-нащадку або підкласу) успадковувати властивості та методи іншого класу (батьківського класу або суперкласу). Наслідування сприяє повторному використанню коду, зменшує його дублювання та організовує класи в ієрархічну структуру. У Java для визначення наслідування використовується ключове слово `extends`. Клас-нащадок може додавати нові властивості та методи, а також перевизначати (`override`) методи, успадковані від батьківського класу.

Поліморфізм (Polymorphism) – це здатність об'єктів різних класів, які мають спільний інтерфейс (наприклад, через наслідування від спільного

батьківського класу або реалізацію спільного інтерфейсу), реагувати на один і той самий виклик методу по-різному. У Java поліморфізм реалізується за допомогою двох основних механізмів:

- **перевантаження методів (Method Overloading):** визначення в одному класі кількох методів з однаковим ім'ям, але різними параметрами (кількістю, типами або порядком). Компілятор визначає, який саме метод викликати, на основі аргументів, переданих при виклику.
- **перевизначення методів (Method Overriding):** визначення в класі-нащадку методу з таким самим ім'ям, списком параметрів та типом повернення, як і у батьківському класі. При виклику такого методу для об'єкта класу-нащадка буде виконано саме версію методу, визначену в класі-нащадку.

Ключовими аспектами поліморфізму є використання посилань на батьківський клас для об'єктів класів-нащадків та використання ключового слова `super` для виклику методів батьківського класу з методів класу-нащадка.

Порядок виконання роботи

1. Відкрийте проект створений для виконання лабораторних робіт у попередніх завданнях.

2. Створіть пакет для даної лабораторної роботи:

- у вікні "Project" (зазвичай зліва) знайдіть каталог `src`;
- клацніть правою кнопкою миші на каталозі `src`;
- оберіть "New" -> "Package";
- введіть назву пакета `lab04`;
- натисніть "OK".

3. Реалізація прикладів наслідування та поліморфізму:

Приклад 1. Наслідування.

- у пакеті `lab04` створіть новий клас `Animal`. Клацніть правою кнопкою миші на пакеті `lab04`, оберіть "New" -> "Java Class" та введіть ім'я `Animal`. Додайте наступний код:

```

package lab04;

public class Animal {
    private String name;

    public Animal(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }

    public void makeSound() {
        System.out.println("Generic animal sound");
    }
}

```

- у пакеті lab04 створіть новий клас Dog, який успадковується від Animal:

```

package lab04;

public class Dog extends Animal {
    public Dog(String name) {
        super(name); // Виклик конструктора батьківського класу
    }

    @Override
    public void makeSound() {
        System.out.println("Woof!");
    }

    public void fetch() {
        System.out.println(getName() + " is fetching the ball.");
    }
}

```

- у пакеті lab04 створіть новий клас Cat, який також успадковується від Animal:

```

package lab04;

public class Cat extends Animal {
    public Cat(String name) {
        super(name);
    }
}

```

```

    }

    @Override
    public void makeSound() {
        System.out.println("Meow!");
    }

    public void purr() {
        System.out.println(getName() + " is purring.");
    }
}

```

Приклад 2: Поліморфізм

- у пакеті lab04 створіть головний клас Main:

```

package lab04;

public class Main {
    public static void main(String args) {
        Animal animal1 = new Animal("Generic Animal");
        Dog dog1 = new Dog("Buddy");
        Cat cat1 = new Cat("Whiskers");

        animal1.makeSound(); // Виведе: Generic animal sound
        dog1.makeSound();    // Виведе: Woof! (Перевизначений метод)
        cat1.makeSound();    // Виведе: Meow! (Перевизначений метод)
        dog1.fetch();        // Виведе: Buddy is fetching the ball.
        cat1.purr();         // Виведе: Whiskers is purring.

        // Поліморфізм через посилання на батьківський клас
        Animal animal2 = new Dog("Lucy");
        Animal animal3 = new Cat("Mittens");

        animal2.makeSound(); // Виведе: Woof! (Викликається перевизначений метод класу
Dog)
        animal3.makeSound(); // Виведе: Meow! (Викликається перевизначений метод класу
Cat)

        // Спроба виклику методу, специфічного для Dog, через посилання на Animal
        // призведе до помилки компіляції
        // animal2.fetch();

        // Перевірка instanceof та приведення типів (якщо потрібно)
        if (animal2 instanceof Dog) {
            Dog dog2 = (Dog) animal2;
            dog2.fetch(); // Тепер можна викликати метод fetch()
        }
    }
}

```

```

        System.out.println("\nДемонстрація перевантаження методів:");
        AnimalShelter shelter = new AnimalShelter();
        shelter.adopt(dog1);
        shelter.adopt(cat1);
        shelter.adopt(animal1);
    }
}

class AnimalShelter {
    public void adopt(Dog dog) {
        System.out.println("Adopting a dog named " + dog.getName());
    }

    public void adopt(Cat cat) {
        System.out.println("Adopting a cat named " + cat.getName());
    }

    public void adopt(Animal animal) {
        System.out.println("Adopting a generic animal named " + animal.getName());
    }
}

```

4. Запуск програми:

- у вікні проєкту відкрийте клас Main.java;
- клацніть правою кнопкою миші на порожньому місці в редакторі коду;
- оберіть "Run 'Main.main()'";
- перегляньте результат виконання програми у вікні "Run".

5. Виконання індивідуального завдання:

- оберіть свій варіант індивідуального завдання (див. нижче);
- створіть необхідні класи та інтерфейси у пакеті lab04 або створіть підпакети, якщо це необхідно для організації коду;
- реалізуйте логіку, використовуючи принципи наслідування та поліморфізму;
- переконайтеся, що ваша програма компілюється та працює коректно.

Індивідуальне завдання

Кожен варіант передбачає створення ієрархії класів, що відображають певну предметну область, та демонстрацію поліморфізму на прикладі обробки об'єктів цих класів. Для кожного завдання необхідно:

- визначити необхідні атрибути класу;
- створити конструктор для ініціалізації атрибутів;
- реалізувати методи, що виконують задані дії;
- у головному класі Main створити екземпляр розробленого класу та продемонструвати роботу його методів.

Варіанти індивідуальних завдань.

1. Створити ієрархію класів Shape (фігура) з підкласами Circle (коло), Rectangle (прямокутник), Triangle (трикутник). Додати метод calculateArea() (обчислити площу) до кожного класу. Продемонструвати поліморфізм, створивши масив об'єктів Shape та викликавши метод calculateArea() для кожного з них.

2. Створити ієрархію класів Vehicle (транспортний засіб) з підкласами Car (автомобіль), Motorcycle (мотоцикл), Truck (вантажівка). Додати метод startEngine() (запустити двигун) до кожного класу. Продемонструвати поліморфізм.

3. Створити ієрархію класів Employee (працівник) з підкласами Manager (менеджер), Developer (розробник), Tester (тестувальник). Додати метод calculateSalary() (розрахувати зарплату) до кожного класу. Продемонструвати поліморфізм.

4. Створити ієрархію класів Product (продукт) з підкласами Book (книга), ElectronicDevice (електронний пристрій), Clothing (одяг). Додати метод getPrice() (отримати ціну) до кожного класу. Продемонструвати поліморфізм.

5. Створити ієрархію класів Animal (тварина) з підкласами Bird (птах), Fish (риба), Mammal (ссавець). Додати метод move() (переміщатися) до кожного класу. Продемонструвати поліморфізм.

6. Створити ієрархію класів BankAccount (банківський рахунок) з підкласами SavingsAccount (ощадний рахунок), CheckingAccount (поточний рахунок). Додати методи deposit() (поповнити) та withdraw() (зняти) до кожного класу. Продемонструвати поліморфізм.

7. Створити ієрархію класів `MediaItem` (медіа-елемент) з підкласами `Audio` (аудіо), `Video` (відео), `Image` (зображення). Додати метод `play()` (відтворити) до класів `Audio` та `Video`, та метод `display()` (відобразити) до класу `Image`. Продемонструвати поліморфізм.

8. Створити ієрархію класів `Computer` (комп'ютер) з підкласами `Laptop` (ноутбук), `Desktop` (настільний комп'ютер), `Tablet` (планшет). Додати метод `powerOn()` (ввімкнути) до кожного класу. Продемонструвати поліморфізм.

9. Створити ієрархію класів `GeometricShape` (геометрична фігура) з підкласами `Square` (квадрат), `Cube` (куб), `Sphere` (сфера). Додати методи `calculateArea()` та `calculatePerimeter()` (для плоских фігур) або `calculateVolume()` (для об'ємних фігур). Продемонструвати поліморфізм.

10. Створити ієрархію класів `Document` (документ) з підкласами `TextDocument` (текстовий документ), `Spreadsheet` (електронна таблиця), `Presentation` (презентація). Додати метод `open()` (відкрити) до кожного класу. Продемонструвати поліморфізм.

11. Створити ієрархію класів `MusicalInstrument` (музичний інструмент) з підкласами `Guitar` (гітара), `Piano` (піаніно), `Drums` (барабани). Додати метод `playNote()` (зіграти ноту) до кожного класу. Продемонструвати поліморфізм.

12. Створити ієрархію класів `FoodItem` (продукт харчування) з підкласами `Fruit` (фрукт), `Vegetable` (овоч), `Meat` (м'ясо). Додати метод `getCalories()` (отримати калорії) до кожного класу. Продемонструвати поліморфізм.

13. Створити ієрархію класів `Transport` (транспорт) з підкласами `Bus` (автобус), `Train` (потяг), `Airplane` (літак). Додати метод `move()` (рухатися) до кожного класу. Продемонструвати поліморфізм.

14. Створити ієрархію класів `Person` (людина) з підкласами `Student` (студент), `Teacher` (вчитель), `Doctor` (лікар). Додати метод `introduce()` (представитися) до кожного класу. Продемонструвати поліморфізм.

15. Створити ієрархію класів File (файл) з підкласами TextFile (текстовий файл), ImageFile (файл зображення), VideoFile (відеофайл). Додати метод open() (відкрити) до кожного класу. Продемонструвати поліморфізм.

16. Створити ієрархію класів Furniture (меблі) з підкласами Chair (стілець), Table (стіл), Sofa (диван). Додати метод assemble() (зібрати) до кожного класу. Продемонструвати поліморфізм.

17. Створити ієрархію класів Appliance (побутовий прилад) з підкласами Refrigerator (холодильник), WashingMachine (пральна машина), Oven (духовка). Додати метод turnOn() (ввімкнути) до кожного класу. Продемонструвати поліморфізм.

18. Створити ієрархію класів GameCharacter (ігровий персонаж) з підкласами Warrior (воїн), Mage (маг), Archer (лучник). Додати метод attack() (атакувати) до кожного класу. Продемонструвати поліморфізм.

19. Створити ієрархію класів Shape3D (тривимірна фігура) з підкласами Cube, Cylinder (циліндр), Cone (конус). Додати метод calculateVolume() до кожного класу. Продемонструвати поліморфізм.

20. Створити ієрархію класів Error (помилка) з підкласами FileNotFoundException, NetworkError, DatabaseError. Додати метод handleError() (обробити помилку) до кожного класу. Продемонструвати поліморфізм.

21. Створити ієрархію класів Device (пристрій) з підкласами Phone (телефон), Tablet, Laptop. Додати метод operate() (працювати) до кожного класу. Продемонструвати поліморфізм.

22. Створити ієрархію класів Flower (квітка) з підкласами Rose (троянда), Tulip (тюльпан), Sunflower (соняшник). Додати метод smell() (пахнути) до кожного класу. Продемонструвати поліморфізм.

23. Створити ієрархію класів Drink (напій) з підкласами Coffee (кава), Tea (чай), Juice (сік). Додати метод prepare() (приготувати) до кожного класу. Продемонструвати поліморфізм.

24. Створити ієрархію класів Tool (інструмент) з підкласами Hammer (молоток), Screwdriver (викрутка), Wrench (ключ). Додати метод use() (використовувати) до кожного класу. Продемонструвати поліморфізм.

25. Створити ієрархію класів WeatherCondition (погодні умови) з підкласами Sunny (сонячно), Rainy (дощивно), Snowy (сніжно). Додати метод getDescription() (отримати опис) до кожного класу. Продемонструвати поліморфізм.

26. Створити ієрархію класів PaymentMethod (спосіб оплати) з підкласами CreditCard (кредитна картка), DebitCard (дебетова картка), PayPal. Додати метод processPayment() (обробити платіж) до кожного класу. Продемонструвати поліморфізм.

27. Створити ієрархію класів House (будинок) з підкласами Apartment (квартира), Townhouse (таунхаус), Villa (вілла). Додати метод getDescription() до кожного класу. Продемонструвати поліморфізм.

28. Створити ієрархію класів TransportTicket (транспортний квиток) з підкласами BusTicket (квиток на автобус), TrainTicket (квиток на потяг), FlightTicket (авіаквиток). Додати метод getPrice() до кожного класу. Продемонструвати поліморфізм.

29. Створити ієрархію класів Unit (військова одиниця) з підкласами Soldier (солдат), Officer (офіцер), Tank (танк). Додати метод attack() до кожного класу. Продемонструвати поліморфізм.

30. Створити ієрархію класів WebsiteElement (елемент вебсайту) з підкласами Button (кнопка), TextBox (текстове поле), Image. Додати метод display() (відобразити) до кожного класу. Продемонструвати поліморфізм.

Контрольні питання

1. Що таке наслідування в об'єктно-орієнтованому програмуванні? Які переваги воно надає?

2. Як оголосити, що один клас успадковується від іншого в Java?

3. Що таке перевизначення методів (method overriding)? Яке ключове слово використовується для анотації перевизначених методів?

4. Що таке поліморфізм? Які існують види поліморфізму в Java?

5. Як працює поліморфізм через посилання на батьківський клас? Наведіть приклад.

6. Для чого використовується ключове слово `super`?

7. Що таке приведення типів (type casting) об'єктів у контексті наслідування та поліморфізму? Коли необхідно його використовувати?.

Література: [1–9, 14, 15].

Лабораторна робота № 5

Тема. Обробка винятків у Java

Мета: отримати практичні навички обробки виняткових ситуацій у програмах на Java з використанням блоків `try-catch-finally`.

Короткі теоретичні відомості

Винятки (exceptions) в Java – це події, які відбуваються під час виконання програми та порушують її нормальний хід. Обробка винятків є важливим аспектом розробки надійного програмного забезпечення, оскільки дозволяє програмі коректно реагувати на помилки та непередбачувані ситуації.

Основні концепції обробки винятків у Java:

- блок `try`: містить код, в якому може виникнути виняток.
- блок `catch`: містить код, який обробляє виняток певного типу. Може бути кілька блоків `catch` для різних типів винятків.
- блок `finally`: містить код, який виконується завжди, незалежно від того, чи був згенерований виняток у блоці `try`, чи ні. Зазвичай використовується для закриття ресурсів (файлів, з'єднань тощо).
- ключове слово `throw`: використовується для явного генерування винятку.
- ключове слово `throws`: використовується в сигнатурі методу для оголошення про те, що метод може згенерувати певний тип винятку.

- ієрархія винятків: усі винятки в Java є нащадками класу Throwable. Основними гілками є Exception (винятки, які програма може обробити) та Error (серйозні помилки, які зазвичай не обробляються програмою). Винятки поділяються на контрольовані (checked exceptions) та неконтрольовані (unchecked exceptions). Контрольовані винятки повинні бути або оброблені, або оголошені в сигнатурі методу. Неконтрольовані винятки (наприклад, NullPointerException, ArrayIndexOutOfBoundsException) не вимагають обов'язкової обробки.

Порядок виконання роботи

1. Відкрийте проект створений для виконання лабораторних робіт у попередніх завданнях.
2. Створіть пакет для даної лабораторної роботи з іменем lab05.
3. Створення Java-класу в пакеті lab05:
 - правою кнопкою миші клацніть на пакеті lab05;
 - оберіть "New" -> "Java Class";
 - введіть назву класу (наприклад, ExceptionHandlerDemo) та натисніть "Enter".
4. Написання коду з обробкою винятків:
 - відкрийте файл ExceptionHandlerDemo.java та введіть наступний код:

```
package lab05;

import java.util.InputMismatchException;
import java.util.Scanner;

public class ExceptionHandlerDemo {
    public static void main(String args) {
        Scanner scanner = new Scanner(System.in);
        try {
            System.out.print("Введіть перше ціле число: ");
            int num1 = scanner.nextInt();
            System.out.print("Введіть друге ціле число: ");
            int num2 = scanner.nextInt();
            int result = num1 / num2;
            System.out.println("Результат ділення: " + result);
        } catch (ArithmeticException e) {
```

```

        System.err.println("Помилка: Ділення на нуль!");
        System.err.println("Деталі помилки: " + e.getMessage());
    } catch (InputMismatchException e) {
        System.err.println("Помилка: Введено некоректні дані! Будь ласка, введіть ціле
число.");
    } finally {
        System.out.println("Блок finally виконано.");
        scanner.close();
    }

    System.out.println("Програма завершила виконання.");
}
}

```

5. Запуск програми:

- правою кнопкою миші клацніть на файлі `ExceptionHandlerDemo.java` в редакторі коду;
- оберіть `"Run 'ExceptionHandlerDemo.main()"`;
- спробуйте ввести різні значення, включаючи нуль як другий дільник та нечислові значення, щоб побачити роботу блоків `catch`. Зверніть увагу на виконання блоку `finally`.

6. Приклад обробки контрольованого винятку:

- створіть новий клас `FileReaderDemo` в пакеті `lab05`:

```

package lab05;

import java.io.File;
import java.io.FileNotFoundException;
import java.util.Scanner;

public class FileReaderDemo {
    public static void main(String args) {
        try {
            File file = new File("nonexistent_file.txt");
            Scanner fileScanner = new Scanner(file);
            while (fileScanner.hasNextLine()) {
                System.out.println(fileScanner.nextLine());
            }
            fileScanner.close();
        } catch (FileNotFoundException e) {
            System.err.println("Помилка: Файл не знайдено!");
            System.err.println("Деталі помилки: " + e.getMessage());
        }
    }
}

```

```
}  
}
```

- запустіть цей клас та переконайтеся, що виняток `FileNotFoundException` обробляється.

7. Виконання індивідуального завдання:

- оберіть свій варіант індивідуального завдання (див. нижче);
- створіть необхідні класи та інтерфейси у пакеті `lab05` або створіть підпакети, якщо це необхідно для організації коду;
- реалізуйте логіку обробки винятків;
- переконайтеся, що ваша програма компілюється та працює коректно.

Індивідуальне завдання

Для кожного варіанту необхідно створити окремий клас у пакеті `lab05` та продемонструвати обробку відповідного винятку. Для кожного завдання необхідно:

- визначити необхідні атрибути класу;
- створити конструктор для ініціалізації атрибутів;
- реалізувати методи, що виконують задані дії;
- у головному класі `Main` створити екземпляр розробленого класу та продемонструвати роботу його методів.

Варіанти індивідуальних завдань.

1. Ділення на нуль: Написати програму, яка запитує у користувача два цілих числа та виконує їх ділення. Обробіть виняток `ArithmeticException`.

2. Вихід за межі масиву: Створити масив цілих чисел розміром 5. Написати програму, яка намагається отримати доступ до елемента за індексом, введеним користувачем. Обробіть виняток `ArrayIndexOutOfBoundsException`.

3. Некоректний формат числа: Написати програму, яка запитує у користувача рядок та намагається перетворити його в ціле число. Обробіть виняток `NumberFormatException`.

4. `NullPointerException`: Створити об'єкт класу (наприклад, `String`) та присвоїти йому значення `null`. Спробуйте викликати метод цього об'єкта. Обробіть виняток `NullPointerException`.

5. `InputMismatchException` (введення рядка замість числа): Написати програму, яка запитує у користувача ціле число. Обробіть виняток `InputMismatchException`, якщо користувач введе рядок.

6. Ділення на нуль (з використанням `finally`): Повторіть завдання 1, додавши блок `finally`, який виводить повідомлення про завершення операції.

7. Вихід за межі масиву (з використанням `finally`): Повторіть завдання 2, додавши блок `finally`.

8. Спроба відкрити неіснуючий файл: Написати програму, яка намагається відкрити файл за вказаним шляхом. Обробіть виняток `FileNotFoundException`.

9. Перетворення рядка в дійсне число: Написати програму, яка запитує у користувача рядок та намагається перетворити його в дійсне число. Обробіть виняток `NumberFormatException`.

10. Доступ до елемента масиву за від'ємним індексом: Створити масив та спробувати отримати доступ до елемента за від'ємним індексом. Обробіть `ArrayIndexOutOfBoundsException`.

11. Створення масиву від'ємного розміру: Спробуйте створити масив з розміром, введеним користувачем. Обробіть виняток `NegativeArraySizeException`, якщо користувач введе від'ємне число.

12. Спроба запису в файл, доступний лише для читання (за потреби, якщо вивчали роботу з файлами): Написати програму, яка намагається записати дані у файл, який має атрибут "лише для читання". Обробіть відповідний виняток (наприклад, `IOException`).

13. Перетворення рядка "null" в об'єкт: Спробуйте перетворити рядок "null" в об'єкт певного класу (якщо це можливо у вашому контексті). Обробіть можливі винятки.

14. Виклик методу з передачею null аргументу, де це заборонено: Напишіть метод, який приймає не-null аргумент. Викличте цей метод, передавши null. Обробіть NullPointerException.

15. Доступ до неіснуючого ключа в HashMap: Створіть об'єкт HashMap та спробуйте отримати значення за ключем, якого немає в мапі. Обробіть можливі винятки (хоча get() повертає null, можна створити ситуацію, де це призведе до NullPointerException пізніше).

16. Розбір некоректного URL (за потреби, якщо вивчали роботу з URL): Написати програму, яка намагається створити об'єкт URL з некоректного рядка. Обробіть виняток MalformedURLException.

17. Спроба з'єднання з неіснуючим сервером (за потреби, якщо вивчали роботу з мережею): Написати програму, яка намагається встановити з'єднання з неіснуючим сервером. Обробіть виняток ConnectException або подібний.

18. Перетворення рядка в дату некоректного формату (за потреби, якщо вивчали роботу з датами): Написати програму, яка намагається перетворити рядок у об'єкт Date з використанням неправильного формату. Обробіть виняток ParseException.

18. Виконання математичної операції, яка призводить до переповнення (за потреби, якщо вивчали обмеження типів даних): Виконайте операцію з числами, яка може призвести до переповнення. Обробіть можливі винятки (залежить від типу даних та операції).

19. Спроба зчитати дані з потоку, який вже закрито (за потреби, якщо вивчали роботу з потоками): Створіть потік введення/виведення, закрийте його та спробуйте зчитати або записати дані. Обробіть виняток IOException.

20. Використання неініціалізованої змінної (може призвести до помилки компіляції, але якщо обійти, може бути `NullPointerException`): Спробуйте використати об'єктну змінну, якій не було присвоєно значення.

21. Кастинг об'єкта до невідповідного типу: Створіть об'єкт одного класу та спробуйте привести його до об'єкта іншого несумісного класу. Обробіть виняток `ClassCastException`.

22. Спроба клонувати об'єкт, який не підтримує клонування (за потреби, якщо вивчали клонування): Створіть клас, який не реалізує інтерфейс `Cloneable`, та спробуйте клонувати його об'єкт. Обробіть виняток `CloneNotSupportedException`.

23. Виклик методу, який викидає контрольований виняток, без його обробки (змусить компілятор видати помилку, але покажіть концепцію): Напишіть метод, який оголошує про викидання контрольованого винятку, та викличте його з іншого методу без обробки.

24. Обробка декількох винятків у одному блоці `catch` (multi-catch): Напишіть код, який може викинути два різних типи винятків, та обробіть їх в одному блоці `catch`.

25. Вкладені блоки `try-catch`: Створіть програму з вкладеними блоками `try-catch` для обробки різних рівнів винятків.

26. Створення власного класу винятку: Створіть власний клас винятку, успадкувавши його від класу `Exception`, та використайте його у своїй програмі (згенеруйте та обробіть).

27. Використання блоку `finally` для закриття ресурсу: Напишіть програму, яка відкриває файл (імітація) та використовує блок `finally` для його закриття незалежно від того, чи відбулася помилка.

28. Генерування винятку за певної умови: Напишіть програму, яка запитує у користувача число та генерує власний виняток, якщо число менше за 0.

29. Комбіноване завдання: Напишіть програму, яка виконує декілька дій, кожна з яких може призвести до різних винятків (наприклад, зчитування з файлу

та перетворення рядка в число), та обробіть ці винятки відповідними блоками catch.

30. Обробка винятків при роботі з масивом об'єктів: Створіть клас, наприклад, Person з полем name. Створіть масив об'єктів типу Person розміром 3. Спробуйте звернутися до поля name елемента масиву за індексом, який виходить за межі масиву, а також до поля name елемента, який є null. Обробіть відповідні винятки (ArrayIndexOutOfBoundsException та NullPointerException)

Контрольні питання

1. Що таке виняток в Java?
2. Яка роль блоків try, catch та finally в обробці винятків?
3. Чим відрізняються контрольовані та неконтрольовані винятки? Наведіть приклади.
4. Як вручну згенерувати виняток у Java? Яке ключове слово для цього використовується?
5. Яке призначення ключового слова throws у сигнатурі методу?
6. Чи може блок try мати кілька блоків catch? Якщо так, то як Java визначає, який блок catch виконати?
7. В якому випадку виконується блок finally?

Література: [1–9, 14, 15].

Лабораторна робота № 6

Тема. Робота з колекціями (List, Set, Map)

Мета: отримати практичні навички роботи з основними типами колекцій Java (List, Set, Map) для зберігання та обробки даних.

Короткі теоретичні відомості

В Java Collections Framework надає набір інтерфейсів та класів для представлення та маніпулювання групами об'єктів, які називаються колекціями. Основні інтерфейси колекцій включають:

- **List:** Впорядкована колекція елементів, яка може містити дублікати. Елементи мають індекс, що дозволяє отримати доступ до них за позицією. Найбільш поширені реалізації: ArrayList, LinkedList.
- **Set:** Колекція, яка не містить дублікатів елементів. Порядок елементів може бути не визначений (наприклад, у HashSet) або визначений (наприклад, у TreeSet, який сортує елементи, або LinkedHashSet, який зберігає порядок вставки).
- **Map:** Колекція, яка представляє собою набір пар ключ-значення. Ключі повинні бути унікальними. Значення можуть повторюватися. Найбільш поширені реалізації: HashMap, TreeMap (сортує за ключами), LinkedHashMap (зберігає порядок вставки).

Для роботи з колекціями використовуються різні методи, такі як додавання елементів (add, put), видалення елементів (remove), перевірка наявності елемента (contains, containsKey, containsValue), отримання розміру колекції (size), ітерування по елементах тощо.

Порядок виконання роботи

1. Відкрийте проект створений для виконання лабораторних робіт у попередніх завданнях.
2. Створіть пакет для даної лабораторної роботи з іменем lab06.
3. Створення класів для демонстрації роботи з колекціями:
 - у пакеті lab06 створіть три Java-класи: ListExample, SetExample, MapExample. Для цього правою кнопкою миші клацніть на пакеті lab06 та оберіть "New" -> "Java Class". Введіть назву класу та натисніть "OK".
4. Реалізація прикладу роботи з List (ListExample.java):
 - відкрийте файл ExceptionHandlerDemo.java та введіть наступний код:

```
package lab06;

import java.util.ArrayList;
```

```

import java.util.List;

public class ListExample {
    public static void main(String args) {
        // Створення ArrayList
        List<String> colors = new ArrayList<>();

        // Додавання елементів
        colors.add("Червоний");
        colors.add("Зелений");
        colors.add("Синій");
        colors.add("Зелений"); // List допускає дублікати

        // Виведення розміру списку
        System.out.println("Розмір списку: " + colors.size());

        // Доступ до елементів за індексом
        System.out.println("Елемент за індексом 0: " + colors.get(0));

        // Перебір елементів списку
        System.out.println("Елементи списку:");
        for (String color : colors) {
            System.out.println(color);
        }

        // Видалення елемента
        colors.remove("Зелений"); // Видаляє перше входження "Зелений"
        System.out.println("Список після видалення 'Зелений': " + colors);

        // Перевірка наявності елемента
        boolean containsBlue = colors.contains("Синій");
        System.out.println("Список містить 'Синій': " + containsBlue);
    }
}

```

5. Реалізація прикладу роботи з Set (SetExample.java):

```

package lab06;

import java.util.HashSet;
import java.util.Set;

public class SetExample {
    public static void main(String args) {
        // Створення HashSet
        Set<Integer> numbers = new HashSet<>();

        // Додавання елементів
        numbers.add(1);
        numbers.add(2);
    }
}

```

```

numbers.add(3);
numbers.add(2); // Set не допускає дублікати

// Виведення розміру множини
System.out.println("Розмір множини: " + numbers.size());

// Перебір елементів множини
System.out.println("Елементи множини:");
for (Integer number : numbers) {
    System.out.println(number);
}

// Видалення елемента
numbers.remove(3);
System.out.println("Множина після видалення 3: " + numbers);

// Перевірка наявності елемента
boolean containsOne = numbers.contains(1);
System.out.println("Множина містить 1: " + containsOne);
}
}

```

6. Реалізація прикладу роботи з Map (MapExample.java):

```

package lab06;

import java.util.HashMap;
import java.util.Map;

public class MapExample {
    public static void main(String args) {
        // Створення HashMap
        Map<String, String> countryCodes = new HashMap<>();

        // Додавання пар ключ-значення
        countryCodes.put("Україна", "UA");
        countryCodes.put("Сполучені Штати Америки", "US");
        countryCodes.put("Німеччина", "DE");
        countryCodes.put("Україна", "UA"); // Ключі повинні бути унікальними, значення
Оновиться

        // Виведення розміру мапи
        System.out.println("Розмір мапи: " + countryCodes.size());

        // Отримання значення за ключем
        String ukraineCode = countryCodes.get("Україна");
        System.out.println("Код України: " + ukraineCode);

        // Перебір ключів мапи

```

```

System.out.println("Ключі мапи:");
for (String country : countryCodes.keySet()) {
    System.out.println(country);
}

// Перебір значень мапи
System.out.println("Значення мапи:");
for (String code : countryCodes.values()) {
    System.out.println(code);
}

// Перебір пар ключ-значення
System.out.println("Пари ключ-значення:");
for (Map.Entry<String, String> entry : countryCodes.entrySet()) {
    System.out.println(entry.getKey() + ": " + entry.getValue());
}

// Перевірка наявності ключа
boolean hasGermany = countryCodes.containsKey("Німеччина");
System.out.println("Мапа містить ключ 'Німеччина': " + hasGermany);
}
}

```

7. Запуск прикладів: Запустіть кожен з класів (ListExample, SetExample, MapExample) окремо, натиснувши правою кнопкою миші на файлі та обравши "Run". Перегляньте результати виконання в консолі.

8. Виконання індивідуального завдання:

- оберіть свій варіант індивідуального завдання (див. нижче);
- створіть необхідні класи та інтерфейси у пакеті lab06 або створіть підпакети, якщо це необхідно для організації коду;
- реалізуйте логіку роботи з колекціями;
- переконайтеся, що ваша програма компілюється та працює коректно.

Індивідуальне завдання

Виконайте одне із запропонованих нижче завдань, використовуючи відповідний тип колекції (List, Set або Map) на ваш розсуд або згідно з вказівкою викладача. Для кожного завдання необхідно:

- визначити необхідні атрибути класу;

- створити конструктор для ініціалізації атрибутів;
- реалізувати методи, що виконують задані дії;
- у головному класі Main створити екземпляр розробленого класу та продемонструвати роботу його методів.

Варіанти індивідуальних завдань.

1. Створити список цілих чисел та знайти середнє значення всіх елементів.
2. Створити список рядків та вивести всі рядки, що починаються на задану літеру.
3. Створити список об'єктів класу Person (з полями name, age) та відсортувати його за віком.
4. Створити множину випадкових цілих чисел в заданому діапазоні.
5. Створити множину слів з заданого тексту (проігнорувати розділові знаки та регістр).
6. Створити множину унікальних символів з введеного рядка.
7. Створити мапу, де ключами є назви продуктів, а значеннями – їх ціни. Вивести всі продукти, ціна яких вища за задану.
8. Створити мапу, де ключами є імена студентів, а значеннями – їхні оцінки. Знайти студента з найвищою оцінкою.
9. Створити мапу, яка відображає кількість входжень кожної літери в заданому тексті.
10. Створити список випадкових чисел та видалити з нього всі парні числа.
11. Створити список рядків та перетворити всі рядки на верхній регістр.
12. Створити список об'єктів класу Book (з полями title, author) та знайти всі книги заданого автора.
13. Створити множину випадкових чисел та знайти найбільше та найменше значення.
14. Створити множину рядків та перевірити, чи містить вона заданий рядок.

15. Створити множину об'єктів класу Point (з полями x, y) та видалити всі точки, що знаходяться на заданій відстані від початку координат.
16. Створити мапу, де ключами є назви міст, а значеннями – їхні поштові індекси. Вивести всі міста із заданим поштовим індексом.
17. Створити мапу, де ключами є назви товарів, а значеннями – їхня кількість на складі. Збільшити кількість заданого товару на певне значення.
18. Створити мапу, яка відображає частоту зустрічання кожного слова в заданому тексті.
19. Створити список цілих чисел та знайти суму всіх непарних чисел.
20. Створити список рядків та об'єднати їх в один рядок, розділений пробілами.
21. Створити список об'єктів класу Employee (з полями name, salary) та знайти середню зарплату.
22. Створити множину цілих чисел та знайти їхній добуток.
23. Створити множину рядків та відсортувати її в алфавітному порядку.
24. Створити множину об'єктів класу Circle (з полем radius) та знайти коло з найбільшим радіусом.
25. Створити мапу, де ключами є номери телефонів, а значеннями – імена власників. Знайти ім'я власника за заданим номером телефону.
26. Створити мапу, де ключами є назви країн, а значеннями – їхні столиці. Вивести всі країни та їхні столиці.
27. Створити мапу, яка відображає оцінки студентів з різних предметів. Знайти середню оцінку кожного студента.
28. Створити список випадкових чисел та відсортувати його у зворотному порядку.
29. Створити список рядків та видалити всі порожні рядки.
30. Створити список об'єктів класу Product (з полями name, price) та знайти продукт з найнижчою ціною.

Контрольні питання

1. Які основні інтерфейси колекцій існують в Java?
2. У чому полягає різниця між List, Set та Map?
3. Які основні реалізації інтерфейсу List ви знаєте? У чому їх відмінність?
4. Які основні реалізації інтерфейсу Set ви знаєте? У чому їх відмінність?
5. Які основні реалізації інтерфейсу Map ви знаєте? У чому їх відмінність?
6. Які методи використовуються для додавання, видалення та пошуку елементів у колекціях List, Set та Map?
7. Що таке ітератор та як його використовувати для перебору елементів колекції?

Література: [1–9, 14, 15].

Лабораторна робота № 7

Тема. Робота з файлами та серіалізація

Мета: оволодіти практичними навичками роботи з файловою системою та механізмом серіалізації об'єктів у Java.

Короткі теоретичні відомості

В Java для роботи з файлами використовуються класи, що знаходяться в пакеті `java.io`. Основні класи включають:

- `File`: представляє файл або каталог у файловій системі. Дозволяє виконувати операції, такі як створення, видалення, перейменування файлів та каталогів, а також отримання інформації про них (розмір, дата створення тощо).
- `InputStream` та `OutputStream`: абстрактні класи для потоків введення та виведення байтів відповідно.
- `FileInputStream` та `FileOutputStream`: класи для читання та запису байтів з/у файл.
- `Reader` та `Writer`: абстрактні класи для потоків введення та виведення символів відповідно.

- `FileReader` та `FileWriter`: класи для читання та запису символів з/у файл.
- `BufferedReader` та `BufferedWriter`: класи, що надають буферизоване читання та запис символів, що підвищує ефективність операцій введення-виведення.
- `PrintWriter`: клас для зручного форматowanego запису даних у текстовий файл.

Серіалізація – це процес перетворення об’єкта в потік байтів для його збереження (наприклад, у файлі) або передачі по мережі. **Десеріалізація** – це процес відновлення об’єкта з потоку байтів.

Для того щоб об’єкт міг бути серіалізований, його клас повинен реалізовувати інтерфейс `java.io.Serializable`. Цей інтерфейс є маркерним і не містить жодних методів.

Для серіалізації та десеріалізації використовуються класи `ObjectOutputStream` та `ObjectInputStream` відповідно.

Порядок виконання роботи

1. Відкрийте проект створений для виконання лабораторних робіт у попередніх завданнях та створіть у ньому новий пакет для даної лабораторної роботи з іменем `lab07`.

2. Робота з текстовими файлами.

- у пакеті `lab07` створіть новий клас, наприклад, `TextFileOperations`;
- реалізуйте метод для запису рядка у текстовий файл. Використовуйте класи `FileWriter` або `BufferedWriter`:

```
package lab07;

import java.io.BufferedWriter;
import java.io.FileWriter;
import java.io.IOException;

public class TextFileOperations {
    public static void writeToFile(String filename, String text) {
        try (BufferedWriter writer = new BufferedWriter(new FileWriter(filename))) {
            writer.write(text);
        } catch (IOException e) {
```



```

        System.err.println("Помилка запису у файл: " + e.getMessage());
    }
}

public static void main(String args) {
    String filename = "output.txt";
    String content = "Це приклад тексту для запису у файл.\nЩе один рядок.";
    writeToFile(filename, content);
    System.out.println("Дані успішно записано у файл " + filename);
}
}

```

– реалізуйте метод для читання вмісту текстового файлу.

Використовуйте класи `FileReader` або `BufferedReader`:

```

package lab07;

import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;

public class TextFileOperations {
    // ... (попередній метод writeToFile) ...

    public static String readFile(String filename) {
        StringBuilder content = new StringBuilder();
        try (BufferedReader reader = new BufferedReader(new FileReader(filename))) {
            String line;
            while ((line = reader.readLine()) != null) {
                content.append(line).append("\n");
            }
        } catch (IOException e) {
            System.err.println("Помилка читання з файлу: " + e.getMessage());
        }
        return content.toString();
    }

    public static void main(String args) {
        String filename = "output.txt";
        String content = "Це приклад тексту для запису у файл.\nЩе один рядок.";
        writeToFile(filename, content);
        System.out.println("Дані успішно записано у файл " + filename);

        String readContent = readFile(filename);
        System.out.println("\nВміст файлу " + filename + ":\n" + readContent);
    }
}

```

3. Робота з серіалізацією об'єктів:

- у пакеті lab07 створіть клас, об'єкт якого ви будете серіалізувати, наприклад, Student. Обов'язково реалізуйте інтерфейс Serializable:

```
package lab07;
```

```
import java.io.Serializable;
```

```
public class Student implements Serializable {  
    private String name;  
    private int age;  
    private transient String password; // Це поле не буде серіалізоване  
  
    public Student(String name, int age, String password) {  
        this.name = name;  
        this.age = age;  
        this.password = password;  
    }  
  
    public String getName() {  
        return name;  
    }  
  
    public int getAge() {  
        return age;  
    }  
  
    public String getPassword() {  
        return password;  
    }  
  
    @Override  
    public String toString() {  
        return "Student{" +  
            "name=" + name + "\" +  
            ", age=" + age +  
            ", password=" + (password == null ? "приховано" : password) + "\" +  
            '}'";  
    }  
}
```

- створіть клас, наприклад, SerializationDemo, у пакеті lab07 та реалізуйте методи для серіалізації та десеріалізації об'єктів:

```
package lab07;
```

```
import java.io.*;
```

```
public class SerializationDemo {
```

```

    public static void serializeObject(String filename, Object obj) {
        try (ObjectOutputStream oos = new ObjectOutputStream(new
FileOutputStream(filename))) {
            oos.writeObject(obj);
            System.out.println("Об'єкт успішно серіалізовано у файл " + filename);
        } catch (IOException e) {
            System.err.println("Помилка серіалізації об'єкта: " + e.getMessage());
        }
    }

    public static Object deserializeObject(String filename) {
        Object obj = null;
        try (ObjectInputStream ois = new ObjectInputStream(new FileInputStream(filename)))
{
            obj = ois.readObject();
            System.out.println("Об'єкт успішно десеріалізовано з файлу " + filename);
        } catch (IOException | ClassNotFoundException e) {
            System.err.println("Помилка десеріалізації об'єкта: " + e.getMessage());
        }
        return obj;
    }

    public static void main(String args) {
        String filename = "student.ser";
        Student student = new Student("Іван", 20, "secret123");
        serializeObject(filename, student);

        Student deserializedStudent = (Student) deserializeObject(filename);
        System.out.println("Десеріалізований об'єкт: " + deserializedStudent);
    }
}

```

4. Виконання індивідуального завдання. Виконайте завдання відповідно до свого варіанту (див. нижче).

Індивідуальне завдання

Кожен варіант передбачає роботу з певним типом об'єктів та виконання операцій з файлами. Для кожного завдання необхідно:

- визначити необхідні атрибути класу;
- створити конструктор для ініціалізації атрибутів;
- реалізувати методи, що виконують задані дії;
- у головному класі Main створити екземпляр розробленого класу та продемонструвати роботу його методів.

Варіанти індивідуальних завдань.

1. Створити клас Книга (назва, автор, рік видання). Записати інформацію про 5 книг у текстовий файл, кожна книга на новому рядку, поля розділити комами. Прочитати дані з файлу та вивести на екран.

2. Створити клас Автомобіль (марка, модель, рік випуску). Серіалізувати масив з 3 автомобілів у файл. Десеріалізувати масив та вивести інформацію про кожен автомобіль.

3. Створити клас Співробітник (ім'я, посада, зарплата). Записати інформацію про 10 співробітників у текстовий файл, кожне поле на новому рядку для кожного співробітника. Прочитати дані та вивести інформацію про співробітників.

4. Створити клас Товар (назва, ціна, кількість). Серіалізувати об'єкт товару у файл. Десеріалізувати та вивести інформацію про товар.

5. Створити клас Студент (прізвище, ім'я, група). Записати інформацію про 7 студентів у текстовий файл, кожне поле розділити символом табуляції. Прочитати дані та вивести список студентів.

6. Створити клас ГеометричнаФігура (тип, площа). Серіалізувати список з 4 геометричних фігур у файл. Десеріалізувати список та вивести інформацію про кожну фігуру.

7. Створити клас Комп'ютер (процесор, ОЗУ, жорсткий диск). Записати інформацію про 3 комп'ютери у текстовий файл у форматі JSON. Прочитати дані та вивести інформацію про комп'ютери. (Для простоти можна імітувати JSON-формат рядками).

8. Створити клас Телефон (модель, виробник, ціна). Серіалізувати об'єкт телефону у файл. Додати до файлу ще один серіалізований об'єкт телефону. Десеріалізувати обидва об'єкти та вивести їх.

9. Створити клас Фільм (назва, жанр, режисер). Записати інформацію про 6 фільмів у текстовий файл, кожне поле розділити символом "|". Прочитати дані та вивести список фільмів.

10. Створити клас МузичнийАльбом (назва, виконавець, рік випуску). Серіалізувати масив з 2 музичних альбомів у файл. Десеріалізувати масив та вивести інформацію про кожен альбом.

11. Створити клас Клієнт (ім'я, адреса, номер телефону). Записати інформацію про 8 клієнтів у текстовий файл, кожне поле на новому рядку для кожного клієнта. Прочитати дані та вивести інформацію про клієнтів.

12. Створити клас Заовлення (номер заовлення, дата, сума). Серіалізувати об'єкт заовлення у файл. Десеріалізувати та вивести інформацію про заовлення.

13. Створити клас Рецепт (назва, інгредієнти (список), час приготування). Записати назву рецепту та кожен інгредієнт на новому рядку у текстовий файл. Прочитати дані та вивести назву рецепту та список інгредієнтів.

14. Створити клас Продукт (назва, виробник, термін придатності). Серіалізувати список з 5 продуктів у файл. Десеріалізувати список та вивести інформацію про кожен продукт.

15. Створити клас Місто (назва, країна, кількість населення). Записати інформацію про 4 міста у текстовий файл, кожне поле розділити крапкою з комою. Прочитати дані та вивести список міст.

16. Створити клас Планета (назва, маса, діаметр). Серіалізувати об'єкт планети у файл. Додати до файлу ще один серіалізований об'єкт планети. Десеріалізувати обидва об'єкти та вивести їх.

17. Створити клас Вебсайт (назва, URL, опис). Записати інформацію про 9 вебсайтів у текстовий файл, кожне поле на новому рядку для кожного вебсайту. Прочитати дані та вивести інформацію про вебсайти.

18. Створити клас Курс (назва, тривалість, викладач). Серіалізувати масив з 3 курсів у файл. Десеріалізувати масив та вивести інформацію про кожен курс.

19. Створити клас Новина (заголовок, текст, дата публікації). Записати інформацію про 7 новин у текстовий файл, кожне поле розділити символом "#". Прочитати дані та вивести список новин.

20. Створити клас Погода (місто, температура, вологість). Серіалізувати об'єкт погоди у файл. Десеріалізувати та вивести інформацію про погоду.

21. Створити клас Колекція (назва, автор, рік створення). Записати інформацію про 6 колекцій у текстовий файл, кожне поле розділити символом "*". Прочитати дані та вивести список колекцій.

22. Створити клас Тварина (вид, кличка, вік). Серіалізувати список з 4 тварин у файл. Десеріалізувати список та вивести інформацію про кожну тварину.

23. Створити клас Валюта (назва, код, курс до гривні). Записати інформацію про 5 валют у текстовий файл у форматі CSV (Comma Separated Values). Прочитати дані та вивести інформацію про валюти.

24. Створити клас Гра (назва, жанр, розробник). Серіалізувати об'єкт гри у файл. Додати до файлу ще один серіалізований об'єкт гри. Десеріалізувати обидва об'єкти та вивести їх.

25. Створити клас Повідомлення (відправник, отримувач, текст). Записати інформацію про 8 повідомлень у текстовий файл, кожне поле на новому рядку для кожного повідомлення. Прочитати дані та вивести інформацію про повідомлення.

26. Створити клас Проєкт (назва, керівник, термін виконання). Серіалізувати масив з 2 проєктів у файл. Десеріалізувати масив та вивести інформацію про кожен проєкт.

27. Створити клас Стаття (заголовок, автор, журнал). Записати інформацію про 9 статей у текстовий файл, кожне поле розділити символом "@". Прочитати дані та вивести список статей.

28. Створити клас Потяг (номер, станція відправлення, станція прибуття). Серіалізувати об'єкт потяга у файл. Десеріалізувати та вивести інформацію про потяг.

29. Створити клас Інструмент (назва, виробник, ціна). Записати інформацію про 7 інструментів у текстовий файл, кожне поле розділити символом "^". Прочитати дані та вивести список інструментів.

30. Створити клас ЕлектроннаПошта (адреса, тема, вміст). Серіалізувати список з 3 електронних листів у файл. Десеріалізувати список та вивести інформацію про кожен лист.

Контрольні питання

1. Які класи в Java використовуються для роботи з файлами? Наведіть приклади.

2. У чому різниця між байтовими та символьними потоками? Коли краще використовувати кожен з них?

3. Що таке серіалізація та десеріалізація об'єктів в Java? Для чого вони потрібні?

4. Який інтерфейс необхідно реалізувати класу, щоб його об'єкти можна було серіалізувати?

5. Які класи використовуються для серіалізації та десеріалізації об'єктів в Java?

6. Що таке поле transient? Як воно впливає на процес серіалізації?

7. Які винятки можуть виникнути під час роботи з файлами та серіалізацією? Як їх обробляти?

Література: [1–9, 14, 15].

3 КРИТЕРІЇ ОЦІНЮВАННЯ ЗНАНЬ СТУДЕНТІВ

Лабораторні роботи студенти виконують у межах курсу «Прикладне програмування на Java».

Повний перелік лабораторних робіт складається з восьми робіт.

За вісім робіт студент може отримати 40 балів. Максимальна кількість балів, яку можуть отримати студенти за виконання однієї лабораторної роботи, складає 5 балів: 3 – «задовільно», 4 – «добре», 5 – «відмінно».

За кожну лабораторну роботу виставляють:

– 5 балів, якщо студент виконав лабораторну роботу, написав звіт і захистив його, тобто правильно й вичерпно відповів на всі питання. Допускаються незначні неточності під час відповіді на одне з контрольних питань;

– 4 бали, якщо студент виконав лабораторну роботу, написав звіт і захистив його, але мав незначні помилки;

– 3 бали, якщо студент виконав лабораторну роботу, написав звіт і захистив його, але під час захисту не відповів на декілька питань і мав незначні помилки під час оформлення звіту.

Розподіл балів за видами занять, що отримують студенти, наведено у табл. 3.1 розділу 3.

Таблиця 3.1 – Розподіл балів за видами занять

Критерії оцінювання		
Вид роботи	Зміст	Бали
Лекції	Робота на лекціях	20
Завдання з ЛР (виконання, захист)	Лабораторна робота № 1	2,5
	Лабораторна робота № 2	2,5
	Лабораторна робота № 3	2,5
	Лабораторна робота № 4	2,5
	Лабораторна робота № 5	2,5

	Лабораторна робота № 6	2,5
	Лабораторна робота № 7	2,5
	Лабораторна робота № 8	2,5
	Лабораторна робота № 9	2,5
	Лабораторна робота № 10	2,5
	Лабораторна робота № 11	2,5
	Лабораторна робота № 12	2,5
	Лабораторна робота № 13	2,5
	Лабораторна робота № 14	2,5
	Лабораторна робота № 15	2,5
	Лабораторна робота № 16	2,5
Контроль	Тест зі змістового модуля 1	5
	Тест зі змістового модуля 2	5
	Тест зі змістового модуля 3	5
	Тест зі змістового модуля 4	5
	Іспит	20
	Разом балів	100

СПИСОК ЛІТЕРАТУРИ

1. Ратушняк Т. В. Програмування мовою JAVA: практикум: навчальний посібник. Державна фіскальна служба України, Університет державної фіскальної служби України. – Ірпінь, 2017. – 212 с.
2. Sierra K., Bates B. Head First Java, 2nd Edition / K. Sierra, B. Bates – O'Reilly Media Inc., 2005. – 688 p
3. Shildt H. Java: A beginner's Guide, Eighth Edition / H. Shildt – McGraw Hill Professional, 2018 – 720 p.
4. Lowe D. Java All-In-One For Dummies, 6th Edition / D. Lowe – For Dummies, 2014. – 912 с.
5. Bloch J. Effective Java / J. Bloch, Addison-Wesley Professional, 2017. – 375 p.
6. Копитко М.Ф., Іванків К.С. Основи програмування мовою Java: Тексти лекцій. – Львів: Видавничий центр ЛНУ ім. Івана Франка, 2002. – 83 с.
7. Брнакевич І.Є., Вагін П.П. Програмування мовою Java: використання фундаментальних класів: Тексти лекцій. – Львів: Видавничий центр ЛНУ імені Івана Франка, 2002. – 75 с.
8. Schildt H. Java: The Complete Reference: 11th Edition, McGraw-Hill Education, 2018, 1208 p.
9. Introduction to Programming Using Java [Electronic resource] – Access mode: <https://math.hws.edu/javanotes/> [in English]
10. Java Application Development on Linux [Electronic resource] – Access mode: <http://javainlinuxbook.com/html/downloads.html> [in English].
11. Microservices for Java Developers [Electronic resource] – Access mode: <https://www.oreilly.com/library/view/microservices-for-java/9781492042228/> [in English].
12. Modern Java EE Design Patterns [Electronic resource] – Access mode: <https://www.oreilly.com/library/view/modern-java-ee/9781492042266/> [in English].

13. Mamchur D., Yatsiuk R. Designing a Neural Adaptive Control System by Method of Analytical Design of Aggregated Nonlinear Regulator for Control an Induction Motor // Proceedings of the 20th IEEE International Conference on Modern Electrical and Energy Systems, MEES 2021, 2021
14. Zagirnyak M., Mamchur D., Gladyr A. DComFra DC-office as a Tool for Digital Skills Development for the Electrical Engineering Students // Proceedings of the 20th IEEE International Conference on Modern Electrical and Energy Systems, MEES 2021, 2021
15. Zagirnyak M., Gladyr A., Mamchur D., Nozhenko V. Peculiarities of Blended Learning Laboratory Workshop for Electrical Engineering Students // Proceedings of the 2022 IEEE 4th International Conference on Modern Electrical and Energy System, MEES 2022, 2022..

Інформаційні ресурси

16. git. URL: <https://git-scm.com/downloads> (дата звернення: 14.07.2023).
17. git (укр.). URL: <https://git-scm.com/book/uk/v2> (дата звернення: 14.07.2023).
18. Docker for Java Developers [Electronic resource] – Access mode: <https://www.oreilly.com/library/view/docker-for-java/978149204262> [in English].
19. Java: The Legend [Electronic resource] – Access mode: https://www.oreilly.com/library/view/java-the-legend/9781492048299/?intcmp=il-prog-free-product-lgen_java_legend [in English].
20. Think Java [Electronic resource] – Access mode: <https://greenteapress.com/thinkjava6/thinkjava.pdf>

Зразок оформлення титульної сторінки звіту

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
КРЕМЕНЧУЦЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ МИХАЙЛА ОСТРОГРАДСЬКОГО

Кафедра комп'ютерної інженерії та електроніки

ЛАБОРАТОРНА РОБОТА
з навчальної дисципліни
«Об'єктно-орієнтовне програмування»

Тема «_____»

Здобувач вищої освіти гр. _____ ПІБ _____

Викладач _____ ПІБ _____

Кременчук 20__

Методичні вказівки щодо виконання лабораторних і самостійної робіт з навчальної дисципліни «Об'єктно-орієнтоване програмування» для здобувачів вищої освіти (першого) бакалаврського рівня денної форми навчання зі спеціальності 123 – «комп'ютерна інженерія» освітньо-професійної програми «комп'ютерна інженерія» (частина 1)

Укладач к. т. н., доц. Д. Г. Мамчур

Відповідальний за випуск завідувач кафедри КІЕ, проф. А. Л. Перекрест

Підп. до др. _____ Формат 60x84 1/16. Папір тип. Друк ризографія.

Ум. друк. арк. _____ Наклад _____ прим. Зам. № _____ Безкоштовно.

Редакційно-видавничий відділ
Кременчуцького національного університету
імені Михайла Остроградського
вул. Університетська, 20 м. Кременчук, 39600